

二、工具调用机制：从 ReAct 的 Action 到工业级 Tool Use 协议

核心问题：模型如何通过工具调用操作真实的开发环境？

OODA 定位：Act 环节的基础设施 / 三阶段共用底层

2.1 从 Action 到 Tool Use：模型如何“伸出手”

回到 ReAct：Action 的价值

上一节我们建立了 ReAct 的 TAO 循环：Thought → Action → Observation。Action 是循环中唯一与外部世界产生交互的环节——没有 Action，模型只是在自己脑子里转圈（纯 CoT）；有了 Action，模型能读文件、改代码、跑测试，循环才有了闭环的可能。

但 ReAct 论文本身对 Action 的定义非常简洁——它就是一个文本字符串，比如 `Search[Python off-by-one error]` 或 `Finish[answer]`。论文关心的是“推理和行动要交替”这个范式，至于 Action 具体怎么执行、用什么协议传递、怎么保证格式正确——留给了工程实现。

那么在工业级系统中，Action 到底是怎么发生的？

Turn 级工具调用：Claude Code 的答案

Claude Code 的做法是 **API turn 级工具调用**。这个“turn”是关键词——模型不是在生成文本的中间停下来调工具，而是**输出一条完整的 assistant 消息后**，通过一个信号表示“我需要调工具”，然后等客户端执行完工具，把结果作为下一条 user 消息回传。

具体流程：

用户: "Fix the bug in buggy_code.py"

|

▼

模型输出 assistant 消息:

[text] "Let me read the file."

← Thought

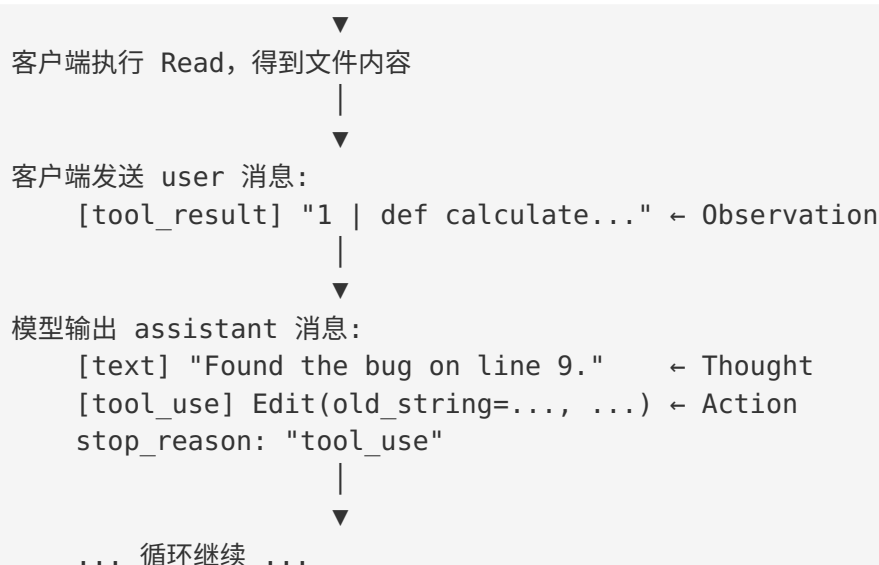
[tool_use] Read(file_path="...")

← Action

stop_reason: "tool_use"

← 信号：我要调工具，请执行后回传结果

|



每一轮 TAO 循环在协议层面就是：一条 assistant 消息（含 text + tool_use blocks）+ 一条 user 消息（含 tool_result blocks）。整洁、可审计、每一步都有完整的请求-响应记录。

关键洞察：循环是 harness 制造的幻觉

这里要揭示 Agent 系统最容易被误解的真相。

很多人看到 Agent 连续执行 Read → Edit → Bash，会以为模型在“持续循环思考”——好像模型有一个内部的 while 循环在驱动整个过程。

事实恰好相反：**模型不知道自己在循环。**

每次调用模型，它只是看到一段对话历史，做一次 next-token prediction，输出一条消息，然后停。它不知道自己处在“第 3 轮 ReAct 循环”——它只知道“我看到了这段对话，我的回复是这样”。

是 harness（客户端）在驱动循环：

```
while True:
    response = call_model(messages)    # 模型只做一次 completion

    if response.stop_reason == "end_turn":
        break                          # 模型说完了 → 退出

    # 模型要调工具 → 执行工具，构造 tool_result，继续循环
    for tool_use in response.tool_uses:
        result = execute_tool(tool_use)
        messages.append(tool_result(result))
```

这段代码就是 Claude Code 的 ReAct 循环驱动器。在我们的 MVP 中，对应 `client.py` 的 `_run_agent_loop` 方法。整个循环逻辑不到 20 行。

CC 源码实证（`query.ts` 行 556-558）：实际工程中，Claude Code 并不信任 `stop_reason` ——源码注释写道 “`stop_reason === 'tool_use' is unreliable - it's not always set correctly`”。CC 用一个 `needsFollowUp` 布尔值替代：只要 `response` 中存在 `tool_use` block 就设为 `true`（行 829-835）。我们的教学简化用 `stop_reason` 做分支是正确的概念模型，但工程实践中应检查 `tool_use` block 的实际存在性，更鲁棒。我们的 MVP 已采纳这个做法。

`stop_reason` 的命名揭示了这个真相。 这个字段叫 “`stop_reason`”（为什么停了），不叫 “`next_action`”（接下来要做什么）。命名视角是推理引擎侧——推理引擎不知道什么 ReAct、什么 Agent，它只知道 “我在生成 token，然后我停了，原因是 X”。

stop_reason	含义	谁做的决定
"end_turn"	模型生成了结束 token，认为说完了	模型自己
"tool_use"	模型生成了工具调用 block，需要工具结果	模型自己
"max_tokens"	撞到 token 上限，被截断	外部约束

Harness 根据 `stop_reason` 做分支：`tool_use` → 继续循环；`end_turn` → 退出。就这么简单。Agent 循环的全部智能在模型侧，全部控制流在 harness 侧。

这不是实现细节——这是架构的核心。因为控制流在 harness 侧，所以：- 换模型不需要改循环逻辑（任何能输出 `tool_use` 格式的模型都行）- 加安全策略只需在 harness 做拦截（模型无法绕过）- 我们的 MVP 能用 Qwen + adapter 层模拟出完全相同的行为

CC 源码实证：这个 “简单的 while 循环” 在生产中并不简单。CC 的 `queryLoop()`（`query.ts` 行 307-1728，共 1400+ 行）包含 **12 个退出点和 7 个继续点**——处理 `prompt-too-long` 恢复、`max_output_tokens` 重试（注入 “Resume directly, no recap” 消息，最多 3 次）、`reactive compact`（紧急上下文压缩）、`stop hooks`（用户配置的 shell 检查点）、`token budget` 续写等。核心循环的骨架虽然简单（检查 `tool_use` → 执行 → 回传），但生产级 harness 需要大量的边界条件处理。这说明 **harness 的工程价值远超 “一个 while 循环”**——它是 Agent 可靠性的基石。

Toolformer：学术界走的另一条路

在 Claude 的 turn 级方案之前，学术界探索了另一条路线。

2023 年，Meta 的 Toolformer 论文提出了 **token 级工具调用**。它的创新分为两部分：训练时的**自监督数据生成**和推理时的**解码循环中断**。

训练阶段：如何让模型“自学”使用工具

Toolformer 的核心洞察是：**不需要人工标注“哪里该用工具”——模型可以自己发现。**

训练流程如下（以论文 Figure 2 的 Pittsburgh 为例）：

原始语料（已有完整文本）：

"Pittsburgh is also known as the Steel City."

Step 1: 采样 — 在每个位置计算"这里需要 API 调用吗？"

模型在 "also known as" 后面给出了高概率 → 这里可能需要查询

Step 2: 生成候选 API 调用

c1 = QA("What other name is Pittsburgh known by?")

c2 = QA("Which country is Pittsburgh in?")

Step 3: 执行 API 调用

r1 = "Steel City"

r2 = "United States"

Step 4: 损失过滤 — 核心筛选机制

对于 c1: $L^- - L^+ \geq \tau f$ ✓ 保留

| L^+ = 给模型看 [QA(...) → Steel City] 后，预测原文 "the Steel City" 的 loss（很低，因为答案就在眼前）

| L^- = 不给任何 API 结果时，预测 "the Steel City" 的 loss（较高）

| 差值大 → 说明这个 API 调用提供了有用信息

|

对于 c2: $L^- - L^+ < \tau f$ ✗ 丢弃

| 即使知道 "United States"，对预测下文 "the Steel City" 也没帮助

Step 5: 将保留的 API 调用织入原文，用于微调

"Pittsburgh is also known as [QA("What other name is...?") → Steel City]
the Steel City."

关键点：**训练时原文是完整的，答案是已知的。**系统用“API 结果能否降低对已知下文的预测 loss”作为筛选标准——这是一个纯自监督信号，不需要任何人工标注。阈值 τf 控制质量：只有确实有用的 API 调用才会被保留进训练数据。

推理阶段：解码循环的中断与恢复

模型微调后，在推理时会自发地在需要的位置生成 API 调用标记。此时就需要**侵入解码循环**来完成“暂停 → 执行 → 注入 → 恢复”的过程：

Step 1: 模型正常自回归生成
"Joe Biden was born in "

Step 2: 模型生成 <API> 标记 → 解码器检测到，继续生成直到产出完整调用
"Joe Biden was born in [QA("Where was Joe Biden born?"))"

Step 3: 解码器暂停，将 API 调用交给外部执行

解码暂停 → QA("Where was Joe Biden born?") → Scranton |

Step 4: 将结果注入生成序列，拼接关闭标记

"Joe Biden was born in [QA("Where was Joe Biden born?") → Scranton]"
↑ 注入真实结果

Step 5: 恢复解码，模型基于包含 API 结果的前缀继续生成

"Joe Biden was born in [QA("Where was Joe Biden born?") → Scranton]
Scranton, Pennsylvania."

“暂停 → 注入 → 恢复”就是 Toolformer 最大的工程负担。你不能用标准的推理 API 实现这个流程——必须修改解码循环：在每一步 token 生成后检查是否触发了 <API> 标记，如果是就暂停自回归、执行外部调用、把结果 token 拼接回 KV cache，然后恢复。这是对推理引擎的侵入式修改。

对比 Claude 的 turn 级方案——完全不需要碰解码循环：

Claude (turn 级) :
Assistant: [text] "Let me look that up." [tool_use] QA("Where was Joe Biden born?")
→ stop_reason: "tool_use" ← 模型正常结束生成，解码循环完整退出
User: [tool_result] "Scranton" ← harness 执行工具，构造新的 API 调用
Assistant: [text] "Joe Biden was born in Scranton, Pennsylvania."
→ stop_reason: "end_turn" ← 又一次正常的、完整的模型调用

类比：- **Toolformer** = 一个人边说话边查手机，查完继续说同一句话——但他的“嘴”（解码器）必须能在说到一半时暂停 - **Claude** = 写完一张便条递给助手，助手执行完回来，再写下一张——每张便条的生成过程是完整的

这两种路线的工程差异是根本性的：

维度	Toolformer (token 级)	Claude (API turn 级)
中断位置	文本生成中间	完整消息结束后
控制流在哪	模型内部的解码循环	harness 外部的 while 循环

维度	Toolformer (token 级)	Claude (API turn 级)
对模型的要求	需要介入解码过程（侵入式）	只需标准 API 调用（非侵入式）
并行工具调用	不支持（单点插入）	天然支持（一条消息多个 tool_use）
可审计性	难（嵌在文本流中）	强（每一步都是独立的 JSON block）
实际采用者	学术研究	几乎所有工业系统

Toolformer 的贡献在于**证明了模型可以学会使用工具**——这是认知上的突破。但其 token 级中间插入的方案在工程上有太多限制：需要修改解码循环、不支持并行调用、难以做权限控制。所以工业界几乎全部选择了 turn 级路线。

一个有趣的历史注脚：Toolformer 论文发表于 2023 年初（基于 GPT-J 6.7B），Claude 的 tool_use API 发布于 2024 年。从 Toolformer 的“模型可以自学使用工具”到 Claude 的“模型通过 API 协议使用工具”，中间的演进不是模型变强了，而是**协议设计成熟了**——把工具调用从模型内部的 token 操作，提升为应用层的结构化协议。Toolformer 的训练范式（自监督发现工具使用时机）至今仍有影响，但其推理时的解码中断方案已被 turn 级协议彻底取代。

2.2 Action Space 设计：四个流派与两条工业路线

理解了工具调用的底层协议后，下一个问题是：**给模型哪些工具？**这就是 Action Space 设计——决定模型能做什么、不能做什么。

不同论文对此给出了截然不同的答案，按设计哲学可以归为四个流派。

流派一：定制命令派 — SWE-agent

SWE-agent (Yang et al., Princeton, 2024) 提出了 **ACI (Agent-Computer Interface)** 的概念：不要让 Agent 直接面对裸终端，而是给它设计一层专用接口。

SWE-agent 的 Action Space 是 **12 个定制 shell 命令**：

```
open <file>           # 打开文件（带行号显示、窗口化）
scroll_down/up        # 翻页（不是一次 cat 整个文件）
goto <line>            # 跳转到指定行
search_file <text>     # 在当前文件中搜索
search_dir <text>      # 在目录中搜索
find_file <name>       # 按文件名搜索
edit <start>:<end>      # 编辑指定行范围
```

```
create <file>      # 创建新文件
submit            # 提交最终 patch
```

设计理念：模拟人类在编辑器中的操作。 人类程序员不会 `cat` 一个 500 行文件然后在脑海中搜索——他们会打开文件、翻到相关部分、搜索关键词。SWE-agent 把这种交互模式编码为 Agent 的接口。

关键的 ACI 设计决策： - **窗口化显示**： `open` 一次只显示 100 行，不是整个文件（控制信息粒度） - **搜索优先**： 提供多种搜索命令，引导模型先定位再阅读（而非盲目翻看） - **行号编辑**： `edit` 用行号范围定位（后来的系统证明了这不是最优选择）

SWE-agent 论文的核心贡献不是这 12 个命令本身，而是提出了一个方法论：**接口设计对 Agent 性能的影响，不亚于模型能力本身。**

论文中最有说服力的实验是消融研究：同一个 GPT-4 模型，逐步叠加 ACI 组件，在 SWE-bench 上的性能逐级跃升——

配置	解决率	累积增加的 ACI 组件
GPT-4 + 标准 Linux 命令行	1.7%	无 (cat/grep/sed)
+ 编辑优化	5.3%	替换 sed 为带行号的 edit 命令
+ 搜索优化	8.1%	替换 grep 为带上下文的 search
+ 导航优化	10.2%	添加 open/scroll 导航命令
+ 错误恢复	12.5%	添加 lint 检查和错误提示

同一个模型，从 1.7% 到 12.5%，7 倍差距，唯一变量是接口设计。 每一步叠加的都不是模型能力，而是 ACI 组件：把 `sed` 换成行号编辑，解决率直接翻 3 倍；加上搜索上下文和导航命令，再翻一倍。裸 shell 下的 GPT-4 经常犯这些错误： `cat` 一个 500 行文件后忘记了前面的内容；用 `grep` 搜到 20 个匹配但无法判断哪个是目标；用 `sed` 行号修改代码但行号算错。ACI 通过窗口化、搜索结果精简、编辑前状态显示，系统性地消除了这些失败模式。

Boyd 类比：ACI 就是 F-86 的气泡座舱——同一个飞行员，换一种视野，战斗力完全不同。差距不在“引擎马力”（模型参数量），而在“座舱视野”（接口设计）。

流派二：语义操作派 — AutoCodeRover

AutoCodeRover (Zhang et al., NUS, 2024) 走了一条更激进的路：**用代码的语义结构而非文本位置来定义操作。**

它的 Action Space 是一组 **AST 级工具**：

```
search_class("Calculator")           # 找到 Calculator 类的定义
search_method("calculate_sum")       # 找到 calculate_sum 方法
search_code("numbers[i + 1]")        # 搜索包含这段代码的位置
search_method_in_class("add", "Calc") # 在特定类中找方法
get_code_snippet("calc.py", 10, 20)  # 获取指定行范围的代码
```

设计理念：代码不是文本，是有结构的。 与其让模型用文本搜索找代码（Grep），不如让它用代码的语义单位（类、方法、变量）来定位。这就像给模型装上了“代码 X 光”——它不再需要从文本表面猜测程序结构，而是直接按结构导航。

优势： - 搜索精度高（按类名/方法名定位，而非字符串匹配） - 天然理解代码层级（类 → 方法 → 代码块） - 与 IDE 的 LSP 能力同源（Go to Definition, Find References）

局限与工业选择： - **语言耦合：**AST 解析与语言强绑定——Python 的 AST 工具不能用在 Rust 上。Claude Code 需要一套工具跨所有语言工作，文本级工具（Grep/Glob/Read）是唯一的语言无关方案 - **环境依赖：**AST 解析需要代码能被正确 parse。对于不完整的代码、动态语言的运行时构造、跨文件的隐式引用，AST 经常失效。文本搜索虽然粗糙但永远可用 - **维护成本：**为每种语言维护 LSP/AST 集成是巨大的工程负担。Claude Code 选择了“模型自己理解代码结构”而非“工具提供结构信息”——用模型智能替代工具复杂度 - **覆盖范围：**AutoCodeRover 的工具集主要覆盖了 Localization（定位），但 Repair（修复）和 Validation（验证）需要回到通用工具（文本编辑、shell 执行）。Claude Code 的 7 个工具覆盖了完整的 L-R-V 链路，不需要为不同阶段切换工具集

这里的 trade-off 本质上是：工具的智能 vs 模型的智能。 AutoCodeRover 把代码理解能力放在工具侧（AST 解析），Claude Code 把代码理解能力放在模型侧（靠模型自己理解 Grep 返回的文本），然后用通用的文本工具配合强模型。随着模型能力的提升，后者的策略越来越有吸引力。

流派三：代码即动作派 — CodeAct

CodeAct (Wang et al., UIUC, 2024) 提出了最极端的简化：**不需要专用工具，代码本身就是动作语言。**

```
# 一个 "Action" 就是一段可执行的 Python/Bash 代码：

# 读文件
with open("buggy_code.py") as f:
    content = f.read()
print(content)
```



```
# 搜索代码
import subprocess
result = subprocess.run(["grep", "-rn", "calculate_sum", "."],
    capture_output=True, text=True)
print(result.stdout)

# 修复 bug
content = content.replace("numbers[i + 1]", "numbers[i]")
with open("buggy_code.py", "w") as f:
    f.write(content)
```

设计理念：代码是程序员最自然的动作表达方式。 为什么要把“读文件”抽象为 `Read(file_path="...")`，当模型可以直接写 `open("...").read()`？为什么要设计 12 个定制命令，当 Python 标准库什么都能做？

CodeAct 的论文数据确实支持这个直觉：在通用任务上，代码动作的成功率比 JSON 工具调用高 20%+。原因是代码的组合能力远超离散工具——一段代码可以包含条件判断、循环、错误处理，一次 Action 就能完成多步逻辑。

但这条路线的代价也很明显：

- **Harness 无法做细粒度权限控制**——这不是说代码“不能区分读和删”，而是说 **harness 无法在执行前判断一段代码会做什么**。结构化工具集下，harness 看到 `Read(file="secret.key")` 可以拒绝、看到 `Bash(command="rm -rf /")` 可以拦截——工具名本身就是权限标签。但面对一段 Python 代码，你在执行前无法静态分析它是读文件还是删文件、是访问本地还是发网络请求。Claude Code 的权限模型（Read 允许、Write 需确认、Bash 受限执行）在 CodeAct 架构下根本无法实现

CC 源码实证（`permissions/permissions.ts`，~1487 行）：CC 的权限系统比上述描述更精细——不仅按工具名分级，还支持**内容级规则**，如 `Bash(git *)` 允许所有 git 命令、`Bash(npm publish:*)` 需要确认。8 个规则来源（`user/project/local/flag/policy/cli/command/session`）形成层级覆盖。甚至有 AI 分类器 `auto` 模式——用模型判断操作是否安全。这种精细权限控制的前提就是结构化工具集：每次调用都是一个命名操作 + 结构化参数，harness 可以对任意维度做规则匹配。CodeAct 的一段代码片段无法提供这种粒度。- **格式安全性差**——代码生成的任何语法错误都导致执行失败 - **可审计性低**——这里的“可审计”指的是：事后回看 Agent 的操作记录时，能否快速理解每一步做了什么。结构化工具调用的审计记录是 `[Step 3] Edit(file="app.py", old_string="x+1", new_string="x")`——一行就看懂。CodeAct 的审计记录是一段 50 行的 Python 脚本，你需要读懂代码才能知道它改了什么。在生产环境中，当 Agent 改出了 Bug 需要回溯时，这个差距是决定性的

流派四：无动态动作派 — Agentless

Agentless (Xia et al., UIUC, 2024) 提出了一个反直觉的问题：**如果我们已经知道修 Bug 的步骤是“定位 → 修复 → 验证”，为什么还要让模型在运行时决定下一步做什么？**

它把整个流程硬编码为三步固定流水线，每一步是一次 LLM 调用——但不是工具调用：

步骤 1 (Localization):

给模型看仓库文件列表 → 模型选择相关文件 → 给模型看代码骨架 → 模型选择相关函数和行号
(层层缩小范围：仓库 → 文件 → 函数 → 行)

步骤 2 (Repair):

给模型看定位到的代码片段 → 模型生成 search/replace patch
(高 temperature 采样 N 个候选 patch，用多样性对冲不确定性)

步骤 3 (Validation):

对 N 个候选 patch 跑测试 → 选通过率最高的
(用测试作为自动化的质量过滤器)

设计理念：将“选择做什么”的决策从模型手中收回 harness。在前三个流派中，模型需要做两层决策——**选哪个工具**和**怎么用这个工具**。Agentless 认为第一层决策（选工具）是 Agent 系统中一个重要的失败来源：模型可能在该搜索时选择了编辑，在该跑测试时选择了继续搜索。如果把“做什么”的顺序固定，模型只需要负责每一步内的内容决策（选哪个文件、怎么改代码），决策空间大幅缩小。

Agentless 的实战表现验证了这个思路：在 SWE-bench Lite 上，它用纯流水线（无工具调用、无 Agent 循环）达到了 27.3% 的解决率，超过了早期版本的 SWE-agent。一个没有任何动态行为的系统，跑赢了一个完整的 Agent——这说明 **对于结构已知的问题，过程编排比动态决策更可靠**。

当然，Agentless 的适用范围有明确边界：它只能处理流程已知的任务（如 Bug 修复）。面对开放式的开发任务（“给这个项目加一个 REST API”），没有预定义的流水线可用，必须回到动态 Agent。但它的存在提醒我们：**不是所有问题都需要 Agent**。能用流水线解决的，就不需要付出 Agent 的决策复杂度成本。

从四个流派到两条工业路线

这四个流派看似各走各路，但在工业实践中收敛为两条主线：

路线 A：结构化工具集（Claude Code, SWE-agent 精神的延续）

模型 → 输出结构化的 tool_use JSON → harness 解析并执行 → 结果回传

工具有明确的名称、参数 schema、权限等级。模型的“行动空间”是一个有限的、可枚举的工具列表。每次 Action 都是一个结构化的函数调用。

代表系统：Claude Code, Cursor, Windsurf

路线 B：代码即动作（OpenHands, CodeAct 精神的延续）

模型 → 输出可执行的 Python/Bash 代码 → 沙箱中执行 → stdout/stderr 回传

模型的“行动空间”是整个编程语言。每次 Action 是一段代码片段。没有工具 schema，没有参数约束。

代表系统：OpenHands, Devin

两条路线不是对错之分，是 **控制力 vs 灵活性** 的 trade-off：

维度	路线 A（结构化工具）	路线 B（代码即动作）
权限控制	天然支持（读/写/执行分级）	困难（代码可以做任何事）
格式可靠性	高（JSON schema 约束）	低（任何语法错误都失败）
可审计性	强（每步是命名的工具调用）	弱（需要理解代码语义）
灵活性	受限于工具集设计	无限（代码什么都能表达）
组合能力	每次只能调一个工具	一段代码可完成多步
Token 成本	高（工具 schema 每次都发送）	低（不需要工具定义）
适合的模型	强模型（能可靠生成 JSON）	会写代码的模型

Claude Code 选择了路线 A。接下来我们深入看它的具体设计。

CC 源码实证（src/services/tools/toolOrchestration.ts，行 19-82）：CC 的工具执行不是简单的串行——它对只读工具和写工具做了不同处理：

- 只读工具（Glob, Grep, Read, LSP 等）：并发执行，最多同时 10 个。模型在一条消息中可以同时调用多个搜索工具，它们并行运行，结果一起返回
- 写工具（Edit, Write, Bash 等）：严格串行。哪怕模型在同一条消息中发出了 3 个 Edit 调用，harness 也会逐个执行

这个设计的原因是**文件系统一致性**：两个并行 Edit 可能修改同一个文件，导致 search/replace 的 old_string 匹配失败。只读操作没有副作用，并行执行完全安全，且能显著加速 Localization 阶段。

对比 CodeAct 路线：一段 Python 代码可能同时包含读和写操作（`content = open("a").read(); open("b","w").write(content)`），harness 无法拆分其中的读写依赖——只能整体串行执行，丧失了并行加速 Observe 的机会。

2.3 Claude Code 的工具集：少即是多

设计哲学：通用原语，不是专用操作

Claude Code 的工具集不是 SWE-agent 的 12 个编辑器命令，也不是 AutoCodeRover 的 AST 语义工具。它选择了一个不同的抽象层次：**通用的文件系统和开发环境原语**。

核心工具只有 7 个：

工具	职责	ACI 关键设计
Read	读取文件内容	<code>offset + limit</code> 参数：控制读取范围，防止大文件撑爆上下文
Write	创建新文件	完整内容写入，用于创建而非修改
Edit	修改已有文件	<code>old_string</code> → <code>new_string</code> ：search/replace 范式，内容定位而非行号定位
Grep	搜索文件内容	<code>head_limit</code> 参数：限制返回结果数量，防止搜索结果爆炸
Glob	按模式搜索文件名	文件发现的入口，配合 Grep 做粗→细定位
Bash	执行 shell 命令	通用执行能力，运行测试、安装依赖、任何 shell 能做的事
Agent	生成子 Agent	上下文隔离的并行执行，用于 Agent Team 模式

为什么是这 7 个，不是更多？

SWE-agent 有 12 个命令（`open`, `scroll_up`, `scroll_down`, `goto`, `search_file`, `search_dir`, `find_file`, `edit`, `create`, `submit`...），Claude Code 只需要 7 个就覆盖了相同甚至更多的能力：

SWE-agent: `open + scroll_down + scroll_up + goto` → Claude Code: `Read(offset=, limit=)`

SWE-agent: search_file + search_dir + find_file Glob	→ Claude Code: Grep + Glob
SWE-agent: edit (行号范围) Edit(old_string, new_string)	→ Claude Code:
SWE-agent: create	→ Claude Code: Write
SWE-agent: (无) (通用执行)	→ Claude Code: Bash
SWE-agent: (无) (子 agent)	→ Claude Code: Agent

少即是多。工具越少，模型选错工具的概率越低，决策空间越小，循环转速越快。这正是 Boyd 的洞察：**精简的行动空间降低决策复杂度，从而提升循环转速。**

Edit 的 search/replace 范式：一个行业共识

在所有工具中，**Edit** 的设计最值得深入。

早期的代码修改工具（包括 SWE-agent 的 edit 命令）使用 **行号定位**：

```
edit 9:9
    total += numbers[i] # Fixed
end_of_edit
```

问题是：**模型对数字天然不敏感**。行号 9 和行号 19 在模型看来没有本质区别，而差一行就可能改到完全不相关的代码。更糟的是，如果之前的编辑插入或删除了行，后续的行号全部偏移——Unified Diff 格式（@@ -10,5 +10,6 @@）也有同样的问题。

业界的共识趋势是转向 **search/replace 范式（内容定位）**——用代码原本身做锚点：

```
{
  "old_string": "total += numbers[i + 1] # Bug: off-by-one error",
  "new_string": "total += numbers[i]"
}
```

模型不需要记行号，只需要复制它刚读到的代码原文。这对模型来说容易得多——它擅长文本模式匹配，不擅长数字计算。

有趣的是，Agentless 独立得出了几乎相同的结论，自创了

<<<< SEARCH / ==== / >>>> REPLACE 格式。Claude Code 的 old_string → new_string 和 Agentless 的 search/replace 在精神上完全一致——**殊途同归**。

Claude Code 在此基础上增加了一层硬约束：**如果 old_string 在文件中匹配到多处，直接拒绝执行**，强制模型提供更多上下文来消歧。宁可报错重试，也不悄悄改错地方。这

是 ACI 设计中“安全优先”的典型决策——与其让模型猜对行号（概率性的），不如让模型提供精确的文本锚点（确定性的）。

CC 源码实证（`FileEditTool/utils.ts`）：CC 的 Edit 实现还有两层我们教学中未展示的工程细节：(1) **引号规范化**——`findActualString()` 将 curly quotes 和 straight quotes 统一后匹配，因为模型经常输出直引号而文件用弯引号；(2) **staleness 检查**——编辑前校验文件自上次 Read 后是否被修改，防止并发操作覆盖他人修改。这两点对小模型尤其重要。

Tool Schema 的 token 成本：一个容易忽视的 trade-off

Claude Code 的工具集虽然只有 7 个基础工具，但加上 Agent Team 的 TaskCreate、TaskUpdate、SendMessage 等，总共约 ~10-15 个工具。每个工具的 JSON Schema 定义（名称、描述、参数类型、必选参数…）大约占 500-800 tokens。

这意味着 **每次 API 调用，工具定义都要随请求一起发送**，大约消耗 **5,000-10,000 tokens** 的 context budget。在一个 10 轮的 ReAct 循环中，这些 schema 重复发送 10 次。

Context Budget 分配（每轮）：

工具 schema:	~8K tokens	（固定开销）
系统提示:	~2K tokens	（固定开销）
对话历史:	~?K tokens	（逐轮增长）
模型输出:	~2K tokens	（可变）

总计 $\leq \text{max_model_len}$ （如 16K, 32K, 128K）

这是路线 A 的固有成本——**用 context tokens 换输出可靠性**。路线 B（CodeAct/OpenHands）不发送工具 schema，省了这笔开销，但牺牲了格式约束。

CC 源码实证（`ToolSearchTool/ToolSearchTool.ts`）：CC 用 **ToolSearch 延迟加载** 解决了这个 trade-off。50+ 个工具中，只有 ~9 个核心工具（Read, Edit, Write, Grep, Glob, Bash, Agent, Skill, Brief）的 schema 始终随请求发送。其余 25+ 个工具标记为 `shouldDefer: true`，不占初始 prompt token。模型需要时先调用 ToolSearch 发现并加载目标工具，再执行调用。这是“按需加载”思想在 tool schema 层面的应用——用一次额外的工具调用换取每轮 ~5K tokens 的 context 节省。

随着模型越强（生成越可靠）+ context window 越大（token 越便宜），路线 A 的优势越明显。但在小模型 + 短 context 的场景（比如我们的 MVP 用 MoE 模型 + 32K context），这个 trade-off 需要认真权衡。

2.4 协议对比：三种格式、一个 adapter

三种 tool_use 协议格式

工业界的工具调用协议看起来各不相同，但做的是同一件事。让我们并排对比三种格式：

Claude Messages API（我们的客户端说这种格式）：

```
{
  "role": "assistant",
  "content": [
    {"type": "text", "text": "Let me read the file."},
    {
      "type": "tool_use",
      "id": "toolu_01A09q90qw90lq917835lh9l",
      "name": "Read",
      "input": {"file_path": "/path/to/buggy_code.py"}
    }
  ],
  "stop_reason": "tool_use"
}
```

特点：typed content block 数组，tool_use_id 精确关联请求和结果。

OpenAI Chat Completions API（vLLM 的 OpenAI 兼容接口说这种格式）：

```
{
  "role": "assistant",
  "content": "Let me read the file.",
  "tool_calls": [
    {
      "id": "call_abc123",
      "type": "function",
      "function": {
        "name": "Read",
        "arguments": "{\"file_path\": \"/path/to/buggy_code.py\"}"
      }
    }
  ],
  "finish_reason": "tool_calls"
}
```

特点：content 是纯字符串，tool_calls 是独立数组，arguments 是 JSON 字符串（不是对象）。

Qwen 原生格式（模型实际生成的文本）：

Qwen2.5 (hermes 格式)：

```
Let me read the file.
<tool_call>
{"name": "Read", "arguments": {"file_path": "/path/to/buggy_code.py"}}
</tool_call>
```

Qwen3 (XML 格式)：

```
Let me read the file.
<function=Read>
<parameter=file_path>/path/to/buggy_code.py</parameter>
</function>
```

特点：纯文本流，没有结构化的 JSON 包裹。`<tool_call>` 和 `</tool_call>` 是词表中的专用 token (id 151657/151658)，模型在训练时大量见过这种格式。

为什么需要 Adapter?

看到差异了——我们的客户端说 Claude 协议（对齐 Claude Code 的真实架构），vLLM 说 OpenAI 协议（行业标准），Qwen 模型说自己的原生格式。中间需要一个翻译层。

这就是 `adapter.py` 的角色：

```
Client (Claude 协议) ↔ adapter.py ↔ vLLM (OpenAI 协议) ↔ Qwen 模型
(原生格式)

          ↑
        本质上做的就是
    Anthropic API 基础设施在做的事
```

adapter 在入口方向做两件事：1. **claude_tools_to_qwen()**：将 Claude 的 `input_schema` 格式转为 OpenAI 的 `parameters` 格式 2.

claude_messages_to_openai()：将 Claude 的 content blocks (text, tool_use, tool_result) 转为 OpenAI 的消息格式

在出口方向做一件事：3. **qwen_response_to_claude()**：将 Qwen 的原始文本输出（含 `<tool_call>` 标签）解析为 Claude 的 typed content blocks

Anthropic 的 API 基础设施也在做完全相同的事——只不过 Anthropic 用前沿模型（几乎不出格式错）+ 约束解码（确定性保证），我们用小模型（可能出格式错）+ 鲁棒解析（概率性兜底）。

小模型的格式可靠性问题：parser.py 的四层 fallback

前沿模型生成工具调用格式几乎 100% 正确。但小模型不行——它会犯各种格式错误：

```
错误 1：用代码块包裹而非 XML 标签
```json
{"name": "Read", "arguments": {"file_path": "..."}}
```

错误 2：漏掉 <tool_call> 开标签
{"name": "Read", "arguments": {"file_path": "..."}}
</tool_call>

错误 3：混入 JS 注释
{"file_path": "/tmp/a.py", // target file
 "old_string": "x = 1"}

错误 4：写成 Python 函数调用语法
Read("/path/to/buggy_code.py")

错误 5：用 Python 三引号而非 JSON 字符串
{"old_string": """def foo():
    pass"""}
```

这就是小模型使用结构化工具集的代价——**格式可靠性不足**。前沿模型几乎 100% 生成正确的 JSON 工具调用格式，但小模型会犯各种格式错误。我们用 parser.py 的四层 fallback 策略来应对：

``` 策略 1：XML 标签（最可靠，Qwen 原生格式的主路径）

{ “name” : “Read” , “arguments” :{...}}

策略 2：代码块（小模型常见的替代格式） `json\n{"name":"Read","arguments":{...}}\n`

策略 3：裸 JSON（花括号深度配对提取） { “name” : “Read” , “arguments” :{...}}

策略 4：函数调用语法（最后手段） Read( “/path/to/file.py” ) → 映射到 Read(file\_path= “...” ) ```

每一层 fallback 都会先跑 `_sanitize_json()` 修复常见畸变（去注释、删尾逗号、转三引号），然后尝试解析。只有当上一层全部失败时才尝试下一层。

## 实战教训：从 Qwen2.5 迁移到 Qwen3 的格式差异

我们在迁移演示模型时经历的一个真实案例：

Qwen2.5-Coder 使用 **hermes 格式**：`<tool_call>{"name":"Read","arguments":{"...}}`  
`</tool_call>`

Qwen3-Coder 使用 **XML 格式**：`<function=Read><parameter=file_path>...</parameter></function>`

当我们用 Qwen3-30B-A3B（MoE 模型）替换 Qwen2.5-Coder-7B 时，vLLM 的 tool-call-parser 还在用 `hermes`。`hermes parser` 在 streaming 模式下会拦截 `<tool_call>` 标签——但 Qwen3 的输出里没有这个标签，而是 `<function=Read>` XML 格式。结果：`hermes parser` 拦截了模型输出但无法解析，导致 **content 被吞掉，streaming 返回空内容**。

从用户视角看到的现象是：Agent 第一轮 Read 成功后，第二轮 “No response from model”——看起来像是模型崩了，实际上是 parser 选错了。

修复方法：自动检测模型架构，Qwen3 用 `qwen3_coder parser`，Qwen2.5 用 `hermes parser`。一个自动检测函数，不到 10 行代码，解决了跨模型兼容性问题。

**这个故事是 ACI 设计哲学的活教材**：接口层的一个小错误（选错 parser），可以让一个完全正常的模型看起来“什么都不会”。Boyd 的框架：Observe 环节失灵（harness 看不到模型的输出），整个 OODA 循环就停转了。

---

## 2.5 Demo 2：协议在实战中的样子

### 演示目标

展示工具调用协议的三层转换过程——模型原始输出 → adapter 转换 → 客户端接收的结构化 content blocks。

### 演示方案

#### Part A：两段请求并排对比

左边：普通 chat completion（无工具）

```
curl -s -X POST http://localhost:9981/generate \
-H "Content-Type: application/json" \
-d '{
 "messages": [{"role": "user", "content": "What is 2+2?"}],
 "system": "You are a helpful assistant."
}' | python -m json.tool
```

→ 模型输出纯文本 content block, `stop_reason: "end_turn"`

右边：带工具定义的请求

```
curl -s -X POST http://localhost:9981/generate \
-H "Content-Type: application/json" \
-d '{
 "messages": [{"role": "user", "content": "Read the file
buggy_code.py"}],
 "system": "You are a coding agent. Use tools to complete tasks.",
 "tools": [
 {
 "name": "Read",
 "description": "Read a file from disk. Returns the file content
with line numbers.",
 "input_schema": {
 "type": "object",
 "properties": {
 "file_path": {
 "type": "string",
 "description": "Absolute path to the file to read"
 }
 }
 },
 "required": ["file_path"]
 }
]
}' | python -m json.tool
```

→ 模型输出 text + tool\_use block, `stop_reason: "tool_use"`

**关键观察：**同一个模型，同样的问题，有没有工具定义决定了模型的输出格式。模型不是“学会了调工具”——而是在看到 tools schema 后，知道了可以用 `<tool_call>` 格式请求执行操作。这对应了本节最重要的认知：**工具调用能力不是模型训练出来的“技能”，而是 API 协议提供的“接口”——模型只是学会了按接口约定输出。**

**Part B：协议转换的完整链路**

展示同一个工具调用请求在三层协议中的不同表示，以及 adapter 如何做转换：

```
from adapter import claude_messages_to_openai, qwen_response_to_claude

— 1. 客户端发送的 Claude 格式 —————
claude_msg = {
 "role": "assistant",
 "content": [
 {"type": "text", "text": "Let me read the file."},
 {"type": "tool_use", "id": "toolu_abc123",
 "name": "Read", "input": {"file_path": "buggy_code.py"}}
]
}
print("Claude 格式:", claude_msg)

— 2. adapter 转为 OpenAI 格式 (发给 vLLM) —————
openai_msgs = claude_messages_to_openai([claude_msg])
print("OpenAI 格式:", openai_msgs)
→ tool_calls[0].function.arguments 是 JSON 字符串，不是对象

— 3. 模型原始输出 (Qwen 生成的文本) —————
raw_output = '''Let me read the file.
<tool_call>
{"name": "Read", "arguments": {"file_path": "buggy_code.py"}}
</tool_call>'''
print("Qwen 原始输出:", raw_output)

— 4. adapter 转回 Claude 格式 (返回给客户端) —————
claude_response = qwen_response_to_claude(raw_output)
print("转回 Claude 格式:", claude_response)
→ content blocks: [text, tool_use], stop_reason: "tool_use"
```

**关键观察：**三种格式表达的是完全相同的语义——“我想调用 Read 工具读取 buggy\_code.py”。差异只在结构：Claude 用 typed content blocks + tool\_use\_id 做精确关联；OpenAI 用独立的 tool\_calls 数组 + JSON 字符串参数；Qwen 用 XML 标签包裹的纯文本。adapter 的全部工作就是在这三种结构之间无损转换。

## Part B 延伸：parser 的格式兜底

准备三段格式不同但语义相同的模型输出，现场展示 parser 的鲁棒解析：

```
from parser import parse_tool_calls

格式 1：标准 XML (最可靠, Qwen 原生格式的主路径)
text1 = 'Let me read it.\n<tool_call>\n{"name": "Read", "arguments":\n {"file_path": "buggy_code.py"}}\n</tool_call>'
```

```
格式 2：代码块包裹（小模型常见的替代格式）
text2 = 'Let me read it.\n```json\n{"name":"Read","arguments":\n {"file_path":"buggy_code.py"}}\n```'

格式 3：裸 JSON（最后手段）
text3 = 'Let me read it.\n{"name":"Read","arguments":\n {"file_path":"buggy_code.py"}}'

for i, text in enumerate([text1, text2, text3], 1):
 result = parse_tool_calls(text)
 print(f"格式 {i}: {result}") # 三种格式都解析出 ToolCall(Read,
 {file_path: ...})
```

**关键观察：**前沿模型几乎 100% 输出格式 1。但小模型可能输出三种格式中的任意一种，甚至输出更畸形的变体（带 JS 注释、Python 三引号等）。parser 的四层 fallback 就是在用工程手段弥补小模型的格式可靠性不足——这是选择路线 A（结构化工具集）时必须承担的工程成本。

## Part C: Qwen2.5 → Qwen3 格式差异（迁移案例）

展示同一个 prompt 在两代模型上的原始输出差异：

```
Qwen2.5: <tool_call>{"name":"Read","arguments":{"file_path":"..."}}</tool_call>
Qwen3: <function=Read><parameter=file_path>...</parameter></function>
```

→ 引出问题：“如果你的 parser 只认一种格式会怎样？” → 引出 MoE 调试中遇到的真实故障。

**关键观察：**两个模型在训练时学会了不同的工具调用文本格式，但 vLLM 的 tool-call-parser 必须与模型格式匹配。如果用了错误的 parser（hermes parser 遇到 Qwen3 的 `<function>` 格式），parser 会拦截模型输出但无法解析，导致客户端收到空内容——从用户视角看，模型“什么都不回了”。这是 ACI 设计中一个典型的“接口层故障伪装成模型层故障”的案例。

## 当演示“失败”：Harness 的兜底与 Reflexion 的雏形

用小模型做现场演示，模型大概率不会每次都完美输出工具调用格式。这些“失败”恰好展示了 harness 在整个系统中的核心地位——**模型可以犯错，但 harness 不能放弃。**

三种典型失败场景及 harness 的应对：

- **模型用文本描述了 Read 但没有生成 `<tool_call>` 标签** → 工具定义注入可能有问题（schema 太长被截断，或 chat template 未正确应用）。Harness 检测到

`stop_reason: "end_turn"` 但任务未完成，可以通过 `nudge`（追加提示“Please use the Read tool”）将模型引导回工具调用路径。这是 ACI 中“提示工程”层面的设计问题

- **模型生成了畸形 JSON** → `parser` 的四层 `fallback` 逐层尝试解析。如果四层全部失败（极端情况），`harness` 不会崩溃，而是将解析失败的信息作为 `tool_result(is_error=True)` 回传给模型：

Round 1 – 模型输出：

```
"Let me read it. Read("/tmp/buggy_code.py")"
```

 ← 畸形格式，`parser` 四层全部失败

Harness 构造错误反馈：

```
tool_result: "Error: Could not parse tool call.
Please use the correct format:
<tool_call>{"name": "Read", "arguments": {...}}</tool_call>"
```

Round 2 – 模型看到错误反馈，修正格式：

```
"Let me try again.
<tool_call>{"name": "Read", "arguments": {"file_path": "/tmp/
buggy_code.py"}}</tool_call>"
```

 ← 格式正确，`parser` 成功解析

**这就是 Reflexion 的最小实例：失败 → 错误反馈进入上下文 → 模型基于反馈调整行为 → 重试成功。** 整个过程不需要任何额外的模块——`harness` 的 `while` 循环 + 错误信息回传，自然地实现了从失败中学习。后面第四讲会系统性地展开 Reflexion 机制，但这里已经能看到它的雏形

- **模型直接回答而非调工具** → `stop_reason: "end_turn"` 触发 `nudge`，`harness` 追加提示引导模型使用工具。本质上和上面一样：`harness` 检测到“模型没有按预期行动”，通过构造新的上下文信息来纠偏

**核心认知：Agent 系统的可靠性不取决于模型永远不犯错，而取决于 harness 能否在模型犯错时将其拉回正轨。** 这正是控制流在 `harness` 侧的架构优势——模型的每一次输出都经过 `harness` 的检查和调度，`harness` 有机会在每一步做干预。如果控制流在模型内部（如 Toolformer 的解码循环），这种逐步纠偏就无从实现。

---

## 2.6 本节小结：从论文到工程的认知升级

回到二维认知地图，本节覆盖了 Act 环节的完整脉络：

|      | Localization | Repair      | Validation |
|------|--------------|-------------|------------|
| Act  | ★ Toolformer | ★ CodeAct   |            |
| (本节) | ★ 四流派对比      | ★ Agentless |            |
|      | ★ CC 工具集     | ★ 协议对比      |            |

三个核心收获：

1. **Turn 级 > Token 级**：工业界选择了 API turn 级工具调用，因为控制流在 harness 侧，非侵入式，支持任意模型后端。Toolformer 的学术贡献在于证明了“模型可以用工具”，Claude 的工程贡献在于把它变成了可靠的协议。
2. **Action Space 设计是 ACI 的核心维度**：四个流派（定制命令/语义操作/代码即动作/无动态动作）代表了不同的控制力-灵活性取舍。Claude Code 的“少而通用”策略（~9 个核心工具始终加载 + 25+ 延迟加载）在精简性和表达力之间找到了平衡点，同时用 ToolSearch 机制解决了 schema token 成本问题。
3. **Adapter 层 = 我们自建的 API 基础设施**：Claude 协议 ↔ OpenAI 协议 ↔ Qwen 原生格式的双向转换，本质上就是 Anthropic 基础设施在做的事。区别只是“前沿模型 + 约束解码”vs “小模型 + 鲁棒解析”。

**CC 源码与我们教学内容的关系**：上述三点都已被 CC 开源代码验证。核心 Agent 循环（`query.ts`，1400+ 行）、50+ 工具的分层加载策略（`ToolSearchTool`）、~1500 行的权限系统（`permissions.ts`），这些都是我们教学中简化模型的工程实现。Anthropic 选择开源意味着我们的课程参考对象从“逆向推测”变成了“源码对照”——后续讲解会持续引用 CC 源码作为实证。

下一节的问题是：当 Agent 在 Localization → Repair → Validation 循环中遇到失败——比如改了代码但测试还是报错——它如何从失败中学习而不是陷入死循环？